

**Client** Probir Roy, Assistant Professor  
**Email** probirr@umich.edu  
**Phone** +1 313-583-6620

**Project period** September 2026–April 2027  
**Team / effort** 3–4 students; 600–800 total hours  
**Client meetings** Biweekly; virtual or in person

### Project Need and Goal

Students in computer organization and operating-systems courses must often learn unfamiliar tools—C and assembly toolchains, Make, Docker, QEMU/xv6, debuggers, and circuit simulators—while simultaneously learning difficult systems concepts. Recurring student feedback identifies four connected needs: reproducible environment setup, clearer transitions from concepts to implementation, smaller guided practice activities, and formative feedback early enough to support revision.

The project will design, implement, and test an installable VS Code extension that serves as a single entry point for setup, coding, processor design, simulation, practice, and help. The AI component will not act as an autonomous grader or unrestricted chatbot. It will provide progressive hints grounded in instructor-approved materials and deterministic evidence from compilation, tests, and simulation traces, and it will route unresolved problems to an instructor or teaching assistant.

### Required Product Capabilities

- **Guided environment setup.** Detect supported prerequisites and provision a versioned development container containing the course compiler, libraries, Make, debugger, QEMU/xv6, and testing tools. If a host prerequisite such as Docker or WSL is missing, provide a safe guided setup rather than silently modifying the operating system.
- **Integrated systems workspace.** Let students write, build, run, test, and debug C and assembly code without manually reconstructing toolchain commands or container configuration.
- **Visual processor designer.** Provide a focused canvas for constructing and testing a 4-bit processor from registers, multiplexers, an ALU, memories, control logic, wires, and buses. Designs must be serializable and reproducible.
- **Evidence engine.** Collect compiler diagnostics, unit-test results, execution traces, and processor signals; identify the first observable divergence from expected behavior; and present actionable, rubric-aligned evidence.
- **Guided practice and AI coach.** Deliver short, Magoosh-style concept lessons, diagnostic questions, worked analogies, mastery checks, and three levels of hints. Technical advice must cite a course resource, test result, code location, or trace event.
- **Human-help bridge.** With student approval, package the question, relevant evidence, attempted fixes, and environment details into a concise request for an instructor or TA. The system must minimize collection and external transmission of student code and identity.

### Deliverables and Acceptance Criteria

#### Deliverables

- Installable VS Code extension (VSIX) and documented source repository.
- Versioned course container and setup diagnostics for Windows, macOS, and Linux, with a documented remote fallback.
- One CIS 450 C/xv6 workflow and one CIS 310 4-bit processor-design workflow.
- At least ten instructor-editable micro-lessons, automated checks, and progressive hint paths.
- User, deployment, privacy, and maintenance documentation; automated test suite; mapped pre/post evaluation package; final demonstration and report.

#### Minimum acceptance criteria

- A new user can create and validate a course workspace through one guided workflow once the supported container runtime is present.
- Reference code and processor designs compile/simulate correctly; seeded faults produce the expected evidence and hint path.
- AI responses are traceable to approved materials or captured evidence, do not assign grades, and clearly escalate unsupported cases.
- Evaluation combines parallel scored assignments, matched questionnaires, telemetry, expert review, and TA workload logs; learning is not inferred from satisfaction alone.

### Team Roles, Schedule, and Client Commitment

The work supports an interdisciplinary team: (1) VS Code/UX, (2) containers and systems integration, (3) processor visualization and simulation, and (4) AI-assisted learning, testing, security, and evaluation. In September–October the team will refine requirements and architecture; November–December will deliver environment provisioning and code execution; January–February will add processor design and evidence-grounded tutoring; March–April will focus on integration, cross-platform testing, usability evaluation, documentation, and final delivery. The client will meet with the team at least biweekly, provide representative course materials and reference tests, answer requirements questions in a timely manner, review milestone demonstrations, and provide the required final product evaluation letter.

**Design boundary:** The minimum product is a focused educational environment for the two reference workflows above, not a general-purpose IDE, circuit simulator, or automated grading system. Additional courses, processor widths, and instructor analytics are stretch goals after the core product is validated.